

## > LAB 1

↳ 6 cells hidden

## > LAB 2

↳ 11 cells hidden

## > LAB 3

↳ 4 cells hidden

## > LAB 4

↳ 20 cells hidden

## > LAB 5

↳ 1 cell hidden

## > LAB 6

↳ 2 cells hidden

## > LAB 7

↳ 6 cells hidden

## ✓ LAB 8

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Model
model = Sequential([
    Dense(10, activation='softmax', input_shape=(784,))
])
```

```

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

model.fit(X_train, y_train, epochs=10, batch_size=64)

test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>

11490434/11490434 — 0s 0us/step

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input\_shape`/`input\_dim`  
 super().\_\_init\_\_(activity\_regularizer=activity\_regularizer, \*\*kwargs)

Model: "sequential"

| Layer (type)  | Output Shape | Param # |
|---------------|--------------|---------|
| dense (Dense) | (None, 10)   | 7,850   |

Total params: 7,850 (30.66 KB)

Trainable params: 7,850 (30.66 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

938/938 — 5s 4ms/step - accuracy: 0.7641 - loss: 0.8852

Epoch 2/10

938/938 — 4s 2ms/step - accuracy: 0.9097 - loss: 0.3323

Epoch 3/10

938/938 — 3s 3ms/step - accuracy: 0.9182 - loss: 0.2959

Epoch 4/10

938/938 — 4s 2ms/step - accuracy: 0.9228 - loss: 0.2810

Epoch 5/10

938/938 — 2s 2ms/step - accuracy: 0.9217 - loss: 0.2799

Epoch 6/10

938/938 — 3s 2ms/step - accuracy: 0.9249 - loss: 0.2676

Epoch 7/10

938/938 — 2s 2ms/step - accuracy: 0.9266 - loss: 0.2666

Epoch 8/10

938/938 — 2s 2ms/step - accuracy: 0.9291 - loss: 0.2547

Epoch 9/10

938/938 — 3s 3ms/step - accuracy: 0.9289 - loss: 0.2607

Epoch 10/10

938/938 — 3s 3ms/step - accuracy: 0.9299 - loss: 0.2557

313/313 — 1s 2ms/step - accuracy: 0.9158 - loss: 0.3004

Test accuracy: 0.925599992275238

with accuracy and loss curves

```

import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import numpy as np
import matplotlib.pyplot as plt

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Model
model = Sequential([
    Dense(10, activation='softmax', input_shape=(784,))
])

model.compile(optimizer='adam',

```

```
        loss='categorical_crossentropy',
        metrics=['accuracy'])

model.summary()

# Train model and store history
history = model.fit(X_train, y_train, epochs=10, batch_size=64)

# Evaluate
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

# -----
# Plot Accuracy Curve
# -----
plt.figure()
plt.plot(history.history['accuracy'])
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.show()

# -----
# Plot Loss Curve
# -----
plt.figure()
plt.plot(history.history['loss'])
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.show()
```

Model: "sequential\_1"

| Layer (type)    | Output Shape | Param # |
|-----------------|--------------|---------|
| dense_1 (Dense) | (None, 10)   | 7,850   |

Total params: 7,850 (30.66 KB)  
Trainable params: 7,850 (30.66 KB)  
Non-trainable params: 0 (0.00 B)

Epoch 1/10  
938/938 11s 9ms/step - accuracy: 0.7760 - loss: 0.8578

Epoch 2/10  
938/938 6s 6ms/step - accuracy: 0.9079 - loss: 0.3374

Epoch 3/10  
938/938 11s 7ms/step - accuracy: 0.9163 - loss: 0.2999

Epoch 4/10  
938/938 2s 2ms/step - accuracy: 0.9203 - loss: 0.2814

Epoch 5/10  
938/938 2s 2ms/step - accuracy: 0.9236 - loss: 0.2755

Epoch 6/10  
938/938 3s 2ms/step - accuracy: 0.9269 - loss: 0.2616

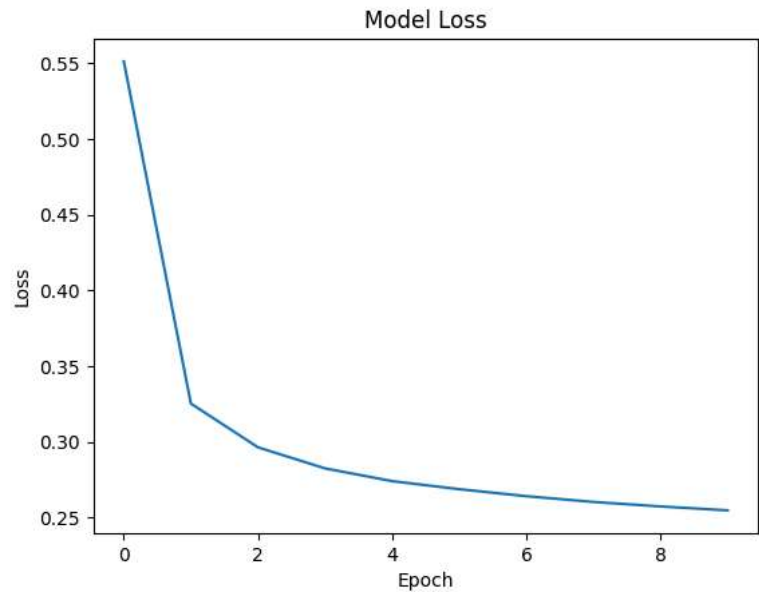
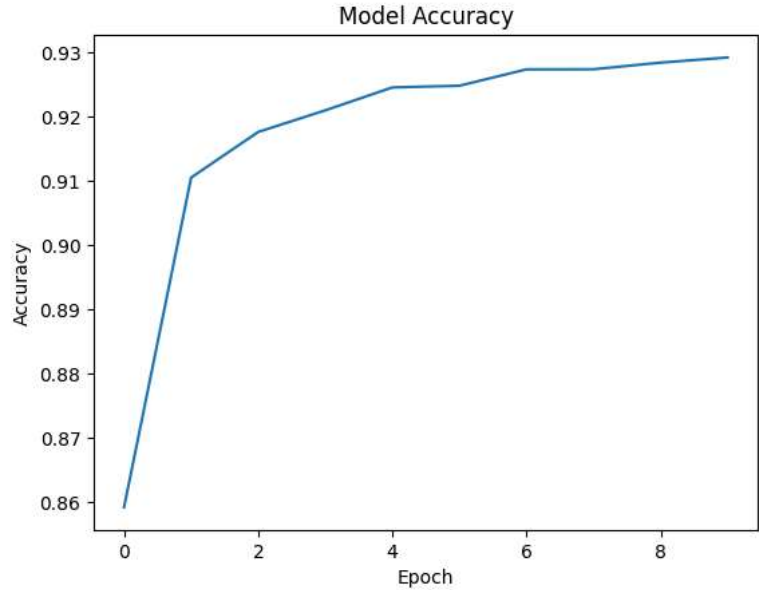
Epoch 7/10  
938/938 2s 2ms/step - accuracy: 0.9265 - loss: 0.2641

Epoch 8/10  
938/938 2s 2ms/step - accuracy: 0.9261 - loss: 0.2626

Epoch 9/10  
938/938 3s 3ms/step - accuracy: 0.9283 - loss: 0.2567

Epoch 10/10  
938/938 3s 3ms/step - accuracy: 0.9297 - loss: 0.2525  
313/313 1s 2ms/step - accuracy: 0.9149 - loss: 0.2977

Test accuracy: 0.9261000156402588



```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt
import numpy as np

# 1. Load MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# 2. Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0

# 3. Flatten images (28x28 → 784)
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# 4. One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# 5. Build model
model = Sequential([
    Dense(10, activation='softmax', input_shape=(784,))
])

# 6. Compile model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# 7. Model summary
model.summary()

# 8. Train model
history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=64,
    validation_data=(X_test, y_test)
)

# 9. Evaluate model
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

# 10. Plot Accuracy Curve
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend(["Train", "Validation"])
plt.show()

# 11. Plot Loss Curve
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend(["Train", "Validation"])
plt.show()
```

Model: "sequential\_18"

| Layer (type)     | Output Shape | Param # |
|------------------|--------------|---------|
| dense_18 (Dense) | (None, 10)   | 7,850   |

Total params: 7,850 (30.66 KB)

Trainable params: 7,850 (30.66 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

938/938 ————— 5s 4ms/step - accuracy: 0.7770 - loss: 0.8615 - val\_accuracy: 0.9110 - val\_loss: 0.3357

Epoch 2/10

938/938 ————— 5s 4ms/step - accuracy: 0.9106 - loss: 0.3310 - val\_accuracy: 0.9172 - val\_loss: 0.2935

Epoch 3/10

938/938 ————— 3s 3ms/step - accuracy: 0.9165 - loss: 0.3004 - val\_accuracy: 0.9218 - val\_loss: 0.2796

Epoch 4/10

938/938 ————— 2s 2ms/step - accuracy: 0.9217 - loss: 0.2830 - val\_accuracy: 0.9226 - val\_loss: 0.2732

Epoch 5/10

938/938 ————— 2s 2ms/step - accuracy: 0.9231 - loss: 0.2743 - val\_accuracy: 0.9245 - val\_loss: 0.2710

Epoch 6/10

938/938 ————— 2s 2ms/step - accuracy: 0.9261 - loss: 0.2694 - val\_accuracy: 0.9240 - val\_loss: 0.2686

Epoch 7/10

938/938 ————— 2s 2ms/step - accuracy: 0.9279 - loss: 0.2608 - val\_accuracy: 0.9250 - val\_loss: 0.2674

Epoch 8/10

938/938 ————— 3s 3ms/step - accuracy: 0.9274 - loss: 0.2620 - val\_accuracy: 0.9263 - val\_loss: 0.2640

Epoch 9/10

938/938 ————— 3s 4ms/step - accuracy: 0.9304 - loss: 0.2518 - val\_accuracy: 0.9257 - val\_loss: 0.2655

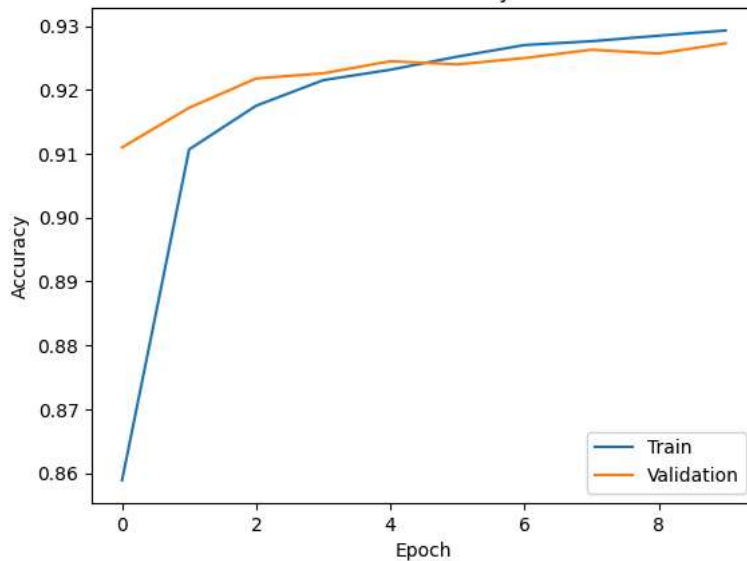
Epoch 10/10

938/938 ————— 4s 2ms/step - accuracy: 0.9309 - loss: 0.2486 - val\_accuracy: 0.9273 - val\_loss: 0.2628

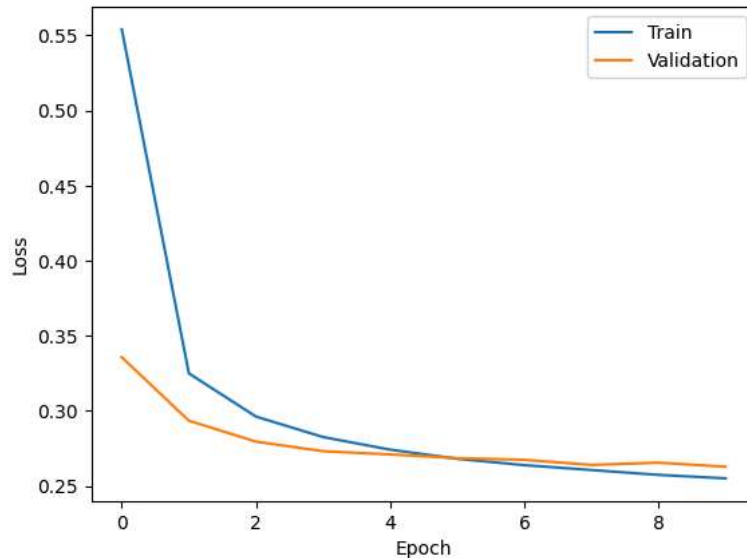
313/313 ————— 1s 2ms/step - accuracy: 0.9160 - loss: 0.2984

Test accuracy: 0.927299976348877

Model Accuracy



Model Loss



```

import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Batch sizes to test
batch_sizes = [32, 64, 128, 256]

for batch in batch_sizes:

    print("Training with batch size:", batch)

    # Model
    model = Sequential([
        Dense(10, activation='softmax', input_shape=(784,))
    ])

    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    # Train model
    history = model.fit(
        X_train,
        y_train,
        epochs=10,
        batch_size=batch,
        validation_data=(X_test, y_test),
        verbose=0
    )

    # Plot graph for this batch
    plt.figure()

    plt.plot(history.history['accuracy'], label='Train Accuracy')
    plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
    plt.plot(history.history['loss'], label='Train Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss')

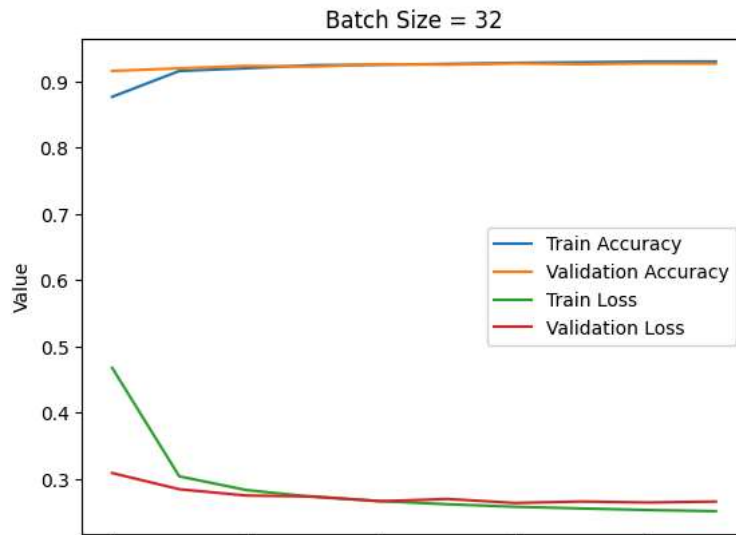
    plt.title(f'Batch Size = {batch}')
    plt.xlabel("Epoch")
    plt.ylabel("Value")
    plt.legend()

    plt.show()

```



Training with batch size: 32



```

import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten images
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Epoch values to test
epoch_values = [10, 15, 20]

for ep in epoch_values:

    print("Training with epochs:", ep)

    # Model
    model = Sequential([
        Dense(10, activation='softmax', input_shape=(784,))
    ])

    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    # Train model
    history = model.fit(
        X_train,
        y_train,
        epochs=ep,
        batch_size=64,
        validation_data=(X_test, y_test),
        verbose=0
    )

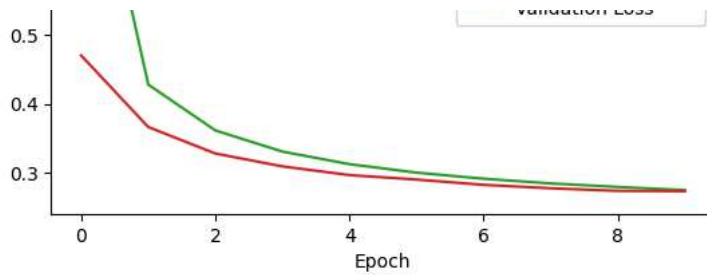
    # Plot graph
    plt.figure()

```

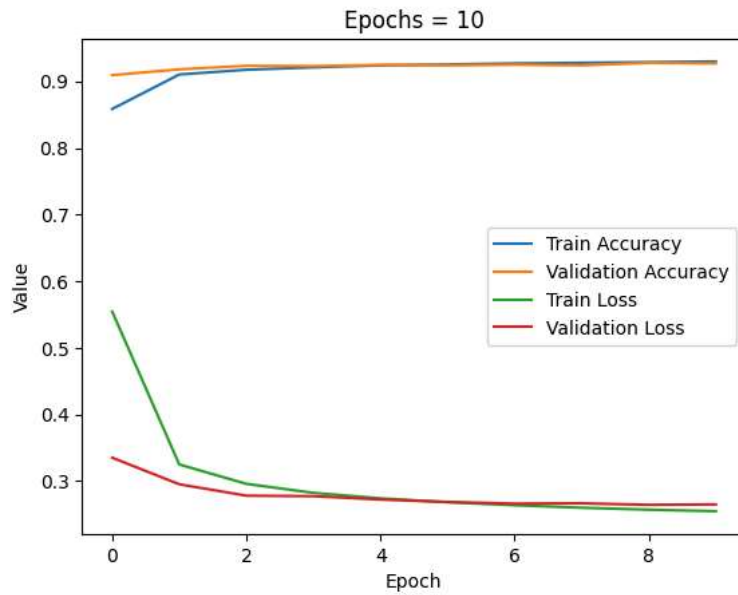
```
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.title(f'Epochs = {ep}')
plt.xlabel("Epoch")
plt.ylabel("Value")
plt.legend()

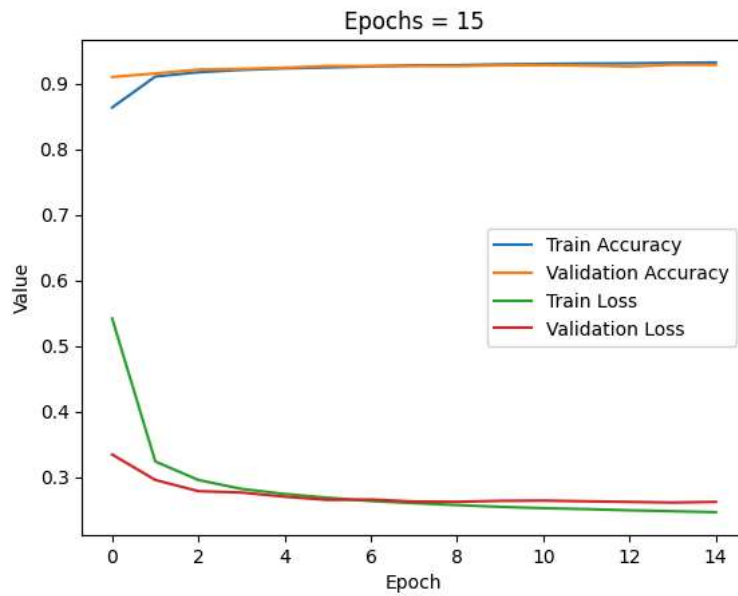
plt.show()
```



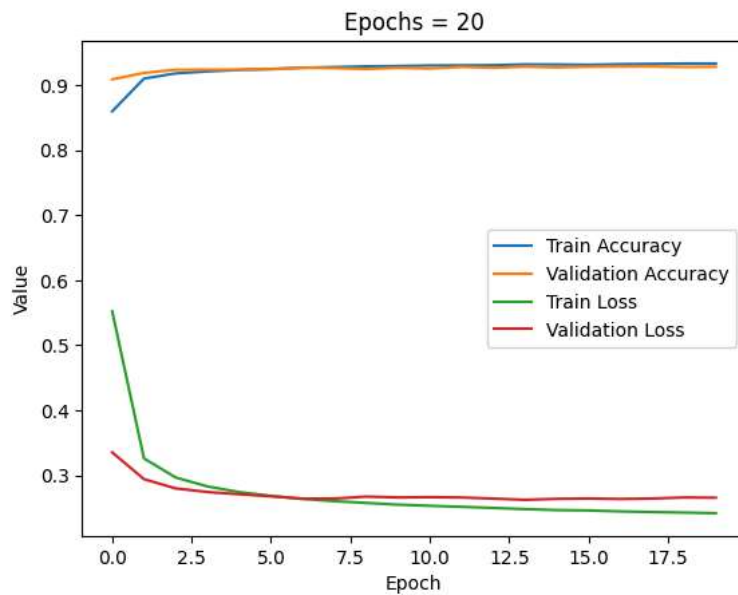
Training with epochs: 10



Training with epochs: 15



Training with epochs: 20



With all the batch sizes

```

import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

batch_sizes = [32, 64, 128, 256]

histories = {}

for batch in batch_sizes:

    print("\nTraining with batch size:", batch)

    model = Sequential([
        Dense(10, activation='softmax', input_shape=(784,))
    ])

    model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

    history = model.fit(
        X_train, y_train,
        epochs=10,
        batch_size=batch,
        verbose=1
    )

    histories[batch] = history

# -----
# Accuracy Graph
# -----
plt.figure()

for batch in batch_sizes:
    plt.plot(histories[batch].history['accuracy'], label=f'Batch {batch}')

plt.title("Training Accuracy for Different Batch Sizes")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

# -----
# Loss Graph
# -----
plt.figure()

for batch in batch_sizes:
    plt.plot(histories[batch].history['loss'], label=f'Batch {batch}')

plt.title("Training Loss for Different Batch Sizes")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()

```



```

Training with batch size: 32
Epoch 1/10
1875/1875 ————— 15s 7ms/step - accuracy: 0.8107 - loss: 0.7195
Epoch 2/10
1875/1875 ————— 6s 3ms/step - accuracy: 0.9116 - loss: 0.3135
Epoch 3/10
1875/1875 ————— 3s 2ms/step - accuracy: 0.9203 - loss: 0.2828
Epoch 4/10
1875/1875 ————— 5s 2ms/step - accuracy: 0.9241 - loss: 0.2731
Epoch 5/10
1875/1875 ————— 4s 2ms/step - accuracy: 0.9276 - loss: 0.2624
Epoch 6/10
1875/1875 ————— 3s 2ms/step - accuracy: 0.9266 - loss: 0.2654
Epoch 7/10
1875/1875 ————— 3s 2ms/step - accuracy: 0.9290 - loss: 0.2556
Epoch 8/10
1875/1875 ————— 4s 2ms/step - accuracy: 0.9288 - loss: 0.2560
Epoch 9/10
1875/1875 ————— 4s 2ms/step - accuracy: 0.9315 - loss: 0.2465
Epoch 10/10
1875/1875 ————— 3s 2ms/step - accuracy: 0.9313 - loss: 0.2519

```

```

Training with batch size: 64
Epoch 1/10
938/938 ————— 2s 2ms/step - accuracy: 0.7677 - loss: 0.8721
Epoch 2/10
938/938 ————— 3s 2ms/step - accuracy: 0.9084 - loss: 0.3341
Epoch 3/10
938/938 ————— 3s 3ms/step - accuracy: 0.9188 - loss: 0.2922
Epoch 4/10
938/938 ————— 3s 3ms/step - accuracy: 0.9217 - loss: 0.2840
Epoch 5/10
938/938 ————— 2s 2ms/step - accuracy: 0.9215 - loss: 0.2789
Epoch 6/10
938/938 ————— 2s 2ms/step - accuracy: 0.9270 - loss: 0.2628
Epoch 7/10
938/938 ————— 3s 2ms/step - accuracy: 0.9258 - loss: 0.2646
Epoch 8/10
938/938 ————— 2s 2ms/step - accuracy: 0.9289 - loss: 0.2546
Epoch 9/10
938/938 ————— 2s 2ms/step - accuracy: 0.9314 - loss: 0.2501
Epoch 10/10
938/938 ————— 3s 3ms/step - accuracy: 0.9277 - loss: 0.2594

```

```

Training with batch size: 128
Epoch 1/10
469/469 ————— 2s 3ms/step - accuracy: 0.7261 - loss: 1.0513
Epoch 2/10
469/469 ————— 1s 2ms/step - accuracy: 0.8978 - loss: 0.3789
Epoch 3/10
469/469 ————— 1s 2ms/step - accuracy: 0.9112 - loss: 0.3260
Epoch 4/10
469/469 ————— 1s 3ms/step - accuracy: 0.9170 - loss: 0.3017
Epoch 5/10
469/469 ————— 1s 2ms/step - accuracy: 0.9208 - loss: 0.2834
Epoch 6/10
469/469 ————— 1s 2ms/step - accuracy: 0.9216 - loss: 0.2811
Epoch 7/10
469/469 ————— 1s 2ms/step - accuracy: 0.9252 - loss: 0.2668
Epoch 8/10
469/469 ————— 1s 2ms/step - accuracy: 0.9289 - loss: 0.2546
Epoch 9/10
469/469 ————— 1s 2ms/step - accuracy: 0.9314 - loss: 0.2501
Epoch 10/10
469/469 ————— 1s 2ms/step - accuracy: 0.9313 - loss: 0.2519

```

Hidden layer 32 (relu)

```

import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt
import numpy as np

# 1. Load MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# 2. Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0

# 3. Flatten images (28x28 → 784)
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# 4. One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

```

```

# 5. Build model (Added hidden layer with 32 neurons)
model = Sequential([
    Dense(32, activation='relu', input_shape=(784,)), # Hidden layer
    Dense(10, activation='softmax')                  # Output layer
])

# 6. Compile model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# 7. Model summary
model.summary()

# 8. Train model
history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=64,
    validation_data=(X_test, y_test)
)

# 9. Evaluate model
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

# 10. Plot Accuracy Curve
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend(["Train", "Validation"])
plt.show()

# 11. Plot Loss Curve
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend(["Train", "Validation"])
plt.show()

```

Epoch



Model: "sequential\_26"

Hidden layer (64-&gt;32)

| Layer (type) | Output Shape | Param # |
|--------------|--------------|---------|
|--------------|--------------|---------|

```

import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt
import numpy as np

# 1. Load MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# 2. Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0

# 3. Flatten images (28x28 → 784)
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# 4. One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# 5. Build model (Added 64 and 32 neuron layers)
model = Sequential([
    Dense(64, activation='relu', input_shape=(784,)), # First hidden layer
    Dense(32, activation='relu'), # Second hidden layer
    Dense(10, activation='softmax') # Output layer
])

# 6. Compile model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# 7. Model summary
model.summary()

# 8. Train model
history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=64,
    validation_data=(X_test, y_test)
)

# 9. Evaluate model
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

# 10. Plot Accuracy Curve
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend(["Train", "Validation"])
plt.show()

# 11. Plot Loss Curve
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend(["Train", "Validation"])
plt.show()

```

Epoch



Model: "sequential\_27"

| Layer (type)                 | Output Shape | Param # |
|------------------------------|--------------|---------|
| Hidden layer (128 → 64 → 32) |              |         |
| dense_28 (Dense)             | (None, 64)   | 50,240  |

```

import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt
import numpy as np

# 1. Load MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# 2. Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0

# 3. Flatten images (28x28 → 784)
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# 4. One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# 5. Build model (Added 128, 64, 32 layers)
model = Sequential([
    Dense(128, activation='relu', input_shape=(784,)), # Hidden layer 1
    Dense(64, activation='relu'), # Hidden layer 2
    Dense(32, activation='relu'), # Hidden layer 3
    Dense(10, activation='softmax') # Output layer
])

# 6. Compile model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# 7. Model summary
model.summary()

# 8. Train model
history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=64,
    validation_data=(X_test, y_test)
)

# 9. Evaluate model
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

# 10. Plot Accuracy Curve
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend(["Train", "Validation"])
plt.show()

# 11. Plot Loss Curve
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")

```

```
plt.legend(["Train", "Validation"])
plt.show()
```



Model: "sequential\_28"

| Layer (type) | Output Shape | Param # |
|--------------|--------------|---------|
|              |              |         |

```

import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt
import numpy as np

# 1. Load MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# 2. Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0

# 3. Flatten images (28x28 → 784)
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# 4. One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# 5. Build model (Added 256 hidden layer)
model = Sequential([
    Dense(256, activation='relu', input_shape=(784,)), # Hidden layer 1
    Dense(128, activation='relu'), # Hidden layer 2
    Dense(64, activation='relu'), # Hidden layer 3
    Dense(32, activation='relu'), # Hidden layer 4
    Dense(10, activation='softmax') # Output layer
])

# 6. Compile model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# 7. Model summary
model.summary()

# 8. Train model
history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=64,
    validation_data=(X_test, y_test)
)

# 9. Evaluate model
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

# 10. Plot Accuracy Curve
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend(["Train", "Validation"])
plt.show()

# 11. Plot Loss Curve
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")

```

```
plt.legend(["Train", "Validation"])
plt.show()
```

Epoch



```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
from google.colab import files

# 1. Load MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# 2. Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# 3. Flatten images
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# 4. One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# 5. Build model
model = Sequential([
    Dense(256, activation='relu', input_shape=(784,)),
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])
```